

```
xiti: docker - Konsole
xiti] at xiti]~$ docker run -it centos bash
[root@2ac70fd88e11 /]# cat /etc/centos-release
CentOS Linux release 7.0.1406 (Core)
[root@2ac70fd88e11 /]# ps
PID TTY          TIME CMD
  1 ?        00:00:00 bash
 18 ?        00:00:00 ps
[root@2ac70fd88e11 /]#
```

As far as this bash session is concerned, it is running on CentOS.

commands in a fresh container based on the image you specify, docker exec runs command in an existing container / layer.

You can see a list of running containers with docker ps and see a list of all running and stopped containers with docker ps -a. Note the container ID and specify it after docker start <id> to start that container, and docker exec <id> <command> to run a command in that container.

Every command you run with docker run crates a new container, even if just listed files with ls. To avoid this you can specify --rm as a command line argument to docker run and it will automatically remove the container when it is done running. This is useful for testing commands or just playing around.

If you make some changes in a container, you can, if you want, save it to an image, so you can base future changes on that. You can have layers on top of layers, so you can have a base Ubuntu image that you download, a layer on top of that that installs a few important software, and another layer on top of that with your own code and files. Or however you see fit.

The great thing here is that creating a new layer does not create a copy of any files, it simply reuses them. So you can have a python layer on top of ubuntu, and a iofs layer on top of ubuntu, and they will both share the same ubuntu install with changes isolated to their respective images.

Terminology

What with layers, and containers and images, it can get a little confusing. Here are some basic Docker terms that will make it much easier to understand what is going on.

- **Images:** These are like ISO or VHD images in the sense that they represent a set of files and folders that constitute an application and its environment. An Ubuntu image will have everything a basic Ubuntu server install has minus the kernel and drivers, and other stuff that interacts with the hardware, since that is handled by the host OS. You can see a list of installed images using docker images.
- **Layers:** If you are familiar with layers in photography software, the effect of layers in Docker is quite similar. For instance, we

*pointers

>>Interesting developer videos



*Talks from GDC 2015

>>The 2015 Game Developers Conference is over and there is a lot to learn from it, there are many free videos that go up on the GDC Vault website after GDC, although many require payment as well.

<http://dggit.in/GDCTalk>



*Microsoft on EdX

>>If you're interested in Microsoft's technologies, they have released a couple of courses with hours upon hours of training video on the brilliant and free EdX platform.

<http://dggit.in/DevwrxEdx>



*Intro to UE4 on GitHub

>>Unreal Engine 4 is now free! So you can now start playing around with the successors of one of the most popular game engines, for free. Here is how you can access its source code on GitHub.

<http://dggit.in/DevUE4Intro>



*Learn Ruby now

>>This video tutorial by Derek Banas tries to teach you the Ruby programming language, covering almost everything.

<http://dggit.in/LearnRubyTut>



have the Debian image, on top of it we add Apache and PHP to get the PHP-Apache image. On top of that we can add a Wordpress install and configuration to get the Wordpress image. Now, you can use the Debian

image in isolation, or the PHP-Apache image, or the Wordpress image. All these images are related to each other but are also usable separately. If you now install Drupal, and it uses the same PHP-Apache image, Docker won't re-download what it already has. Running docker images -a will show you all image layers.

- **Containers:** A container is where you can actually make changes. Whatever commands you run don't affect any of the images; they are read only. Rather any changes are encapsulated inside a container. If your container is useful enough you can make an image of it, and then build containers, or other images on top of that. For instance, you could run an Ubuntu container, install some tools you use often, and then save that to an image.
- **The Docker Registry:** The people behind Docker host a large collection of software that you can install using docker pull. This service is called the Docker registry. You can search it using docker search <search term> or browser it online at <https://registry.hub.docker.com>
- **Dockerfile:** A Dockerfile is text file that serves as a recipe for building images. It starts by defining a base image it will build on, and then installs additional tools, and adds / removes / modifies files in it to get it to the state one wants.

Conclusion

Docker is an immensely useful and powerful tool that has changed the software can be deployed. At just version 1.5 (as of this writing) it has already accumulated powerful tools and features and inspired an entire community of tools and software around it. There are even lightweight operating systems designed around Docker as a means of installing and running software.

Of late Docker has gained even more features such as the ability to orchestrate and entire cluster of Docker containers and link them together. Whether Docker itself is here 10 years from now or not we can't say, but the concepts it has unleashed are here to stay, and are likely to become even more central to Linux in the future.

So head over to docker.com now and figure out how you can get it up and running on your system. >